I. Bölüm - Metin Sınıflandırma

Metin Sınıflandırma Nedir ?

*Metin sınıflandırma (Text classfiction), doğal dilde yazılmış bir metnin içeriğine göre önceden belirlenmiş bir veya birden fazla kategoriye atanması işlemidir.

Örnek

Metin	Sınıf
"Takım maçı 3-0 kazandı."	Spor
"Dolar kuru yükseldi."	Ekonomi
"Yeni işlemci tanıtıldı."	Teknoloji
"Bu ürünü hiç beğenmedim."	Olumsuz

Metin sınıflandırmanın çalışma aşamaları



*Metinler bilgisayar acısından karmaşık yapılardır. Bunun temel nedenleri:

- Bir kelimenin farklı anlamlarda kullanılması
- Kelime sırasının anlam değiştirmesi
- Eş anlamlı kelimelerin farklı yazılması
- İroni mecaz ve uzunlukların farklı olması
- metinlerin uzunluklarının farklı olması
- Eğitim verileri ile gerçek kullanılan verilerin farklılaşabilmesi

Örnek:

"Bankaya para yatırdım."

"Nehir kıyısındaki bankaya oturdum."

Burada *banka* kelimesinin anlamı bağlama göre değişmektedir.

1.2. Matematiksel gösterim

Metin sınıflandırma veri kümesi:

$$D = \{(x_i, y_i)\}_{i=1}^{N}$$

Burada:

- D: Veri kümesi
- x_i: Metin örneği
- y_i: Metnin gerçek sınıf etiketi
- N: Toplam örnek sayısı

Bir sınıflandırma modeli:

$$f_\theta : X \rightarrow Y$$

Modelin amacı, tahmin edilen sınıf ile gerçek sınıf arasındaki hatayı en aza indirmektir.

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} L(f_\theta(x_i), y_i)$$

Burada L, kayıp (loss) fonksiyonudur. θ ise modelin öğrenilebilir parametrelerini temsil eder.

Metin Sınıflandırma

Tür	Açıklama	Örnek
İkili sınıflama	İki farklı sınıf	Spam / Normal
Çok sınıflı	Birden fazla sınıftan yalnızca biri	Spor / Ekonomi / Teknoloji
Çok etiketli	Aynı metne birden çok etiket	Yapay Zeka + Sağlık
Hiyerarşik	Sınıfları alt kategorilere ayır	Teknoloji → Bilgisayar → İşletmeci
Zero-shot	Görev için etiketli eğitim örneği olmadan	Yeni konu etiketi
Few shot	Az sayıda örnek ile	Sınıftan birkaç örnek

Metin Ön İşleme (Text Preprocessing)

* Bilgisayarlar metinleri doğrudan matematiksel olarak hesaplamada kullanmaz. Bu nedenle metinler işlenerek sayısal yapılara dönüştürülmesi gerekir.
* Temel ön işleme yöntemleri:

1. Tokenization: Metnin kelimeleri veya alt kelimelere ayırma
2. Lowercasing: Büyük harfleri küçük harflere dönüştürme
3. Stop-word Removal: Göreve göre bilgi değeri düşük kelimeleri çıkarmak
4. Stemming: Kelimelerin eklerini çıkararak yaklaşık köklerini bulma
5. Lemmatization: Kelimelerin sözlükteki temel biçimlere dönüştürme

Örnek

Girdi metni:

"Öğrenciler bugün robotik projelerini geliştirdiler."

Tokenization sonrası:

Plain text

```
["Öğrenciler", "bugün", "robotik",
 "projelerini", "geliştirdiler"]
```

Küçük harfe dönüştürülmüş hâli:

Plain text

```
["öğrenciler", "bugün", "robotik",
 "projelerini", "geliştirdiler"]
```

Lemmatization yaklaşımıyla örnek temel biçimler:

Plain text

```
["öğrenci", "bugün", "robotik",
 "proje", "geliştirmek"]
```

Not: Ön işleme her model için aynı şekilde yapılmaz. Örneğin BERT gibi Transformer modellerinde modelin kendi tokenizer yapısı kullanılmalıdır.

Metinin Sayısal Temsili

1. One-Hot Encoding

* One-hot encoding, kelimelerin sözlükteki konumlarına göre vektörle gösterilmesidir.

Örneğin:

V = [robot, yapay, zeka, veri]

Dört kelimeli sözcük için:

Kelime	Vektör
robot	[1, 0, 0, 0]
yapay	[0, 1, 0, 0]
zeka	[0, 0, 1, 0]
veri	[0, 0, 0, 1]

* Her kelime kendi konumlarında 1 diğer durumlarda 0 ile ifade edilir.

2. Bag-of-word (BoW)

* Bag of word, metin içerisindeki kelimelerin görünme sayılarına göre temsil eder.
* Bu yöntemde kelime sayısı dikkate alınmaz.

Örnek:

1. metin: "Robot veri işler."

2. metin: "robot robot öğrenir."

Sözlük: V = [robot, veri, işler, öğrenir.]

Vektör:

$$M_1 = [1, 1, 1, 0]$$

$$M_2 = [2, 0, 0, 1]$$

3. TF-IDF (Term frequency - Inverse Document frequency)

TF (Term Frequency):

- Kelimelerin belge içinde sıklığına bağlıdır.

$$TF(t, d) = \frac{f(t, d)}{|d|}$$

$f(t, d)$ → Tekrar sayısı

$|d|$ → Toplam kelime sayısı

Örnek: Robot robot veri işler

$$Tf(robot, d) = \frac{2}{4} = 0.5$$

IDF (Inverse Document Frequency)

Kelimenin belge koleksiyonundaki ayırt ediciliğini ölçer.

$$IDF(t) = \log\left(\frac{N}{df(t)}\right)$$

- N: Toplam belge sayısı
- $df(t)$: Kelimenin geçtiği belge sayısı

TF-IDF formülü

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Örnek:

100 belge içerisinden "robot" kelimesi 10 belgede geçsin.

$$IDF(robot) = \ln(100/10)$$

$$IDF(robot) \approx 2.303$$

TF değeri 0.5 ise:

$$TF\text{-}IDF = 0.5 \times 2.303$$

$$TF\text{-}IDF \approx 1.1515$$

4. N-Gram

* N-gram metindeki ardışık n adet kelimelerin gelmesiyle oluşur.

"Örnek cümle:

"Robot engelli başarıyla ilgilodı."

Tür	
Unigram	robot, engelli, başarıyla, ilgilodı
Bigram	" " , " "
Trigram	" " " , "

Word Embeding (Kelime Görme)

* Embeding, kelimelerin anlam ilişkilerini koruyabilecek şekilde yoğun sayısal vektörlerle temsil edilmesidir.

One-Hot Encoding'den farklı olarak benzer kelimeler vektör uzayında birbirine yakın konumlanabilir.

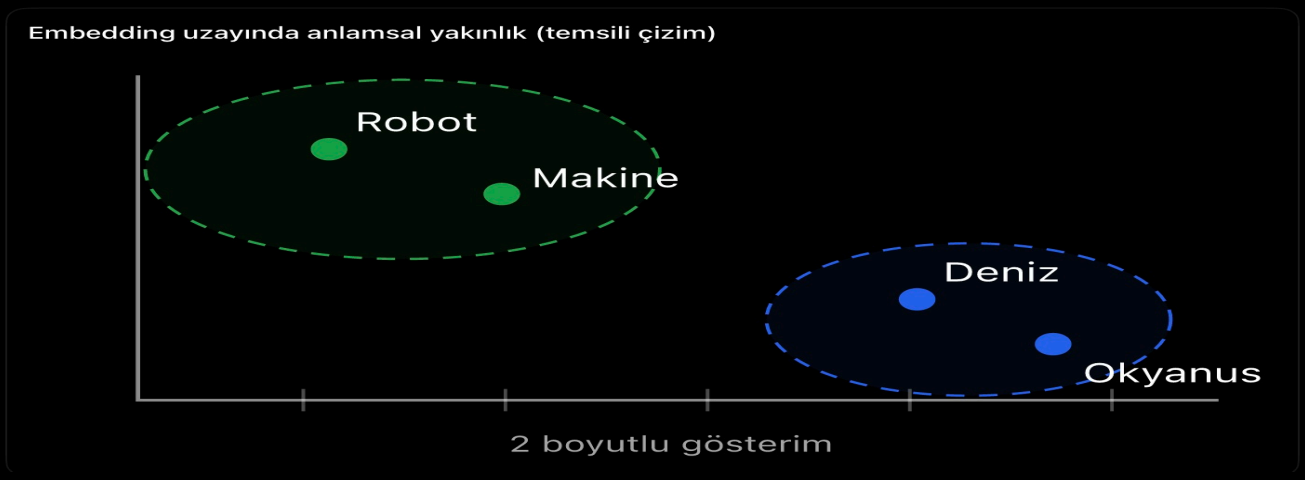
Örnek, yalnızca kavramsal amaçlı:

$$Robot = [0.8, 0.6, 0.2]$$

$$Makine = [0.75, 0.58, 0.25]$$

$$Deniz = [0.1, 0.2, 0.9]$$

Robot ve makine vektörleri birbirine daha yakın olabilir.



Klasik Metin Sınıflandırma Algoritmaları

1. Naive Bayes

* Naive Bayes, bayes teoremine dayanan olasılıksal sınıflandırma algoritmasıdır.
* Temel varsayım, özelliklerin sınıf bilgilerini birbirinden koşulu bağımsız olduğudur.
* Bayes Teorisi:

$$P(C|x) = \frac{P(x|C) \cdot P(C)}{P(x)}$$

- $P(C|x)$: metin verildiğinde sınıfın olasılığı
- $P(x|C)$: Sınıfa bağlı metin "özellikleri "
- $P(C)$: Sınıfın "önsel olasılığı
- $P(x)$: Metnin olasılığı

2. Logistik Regresyon

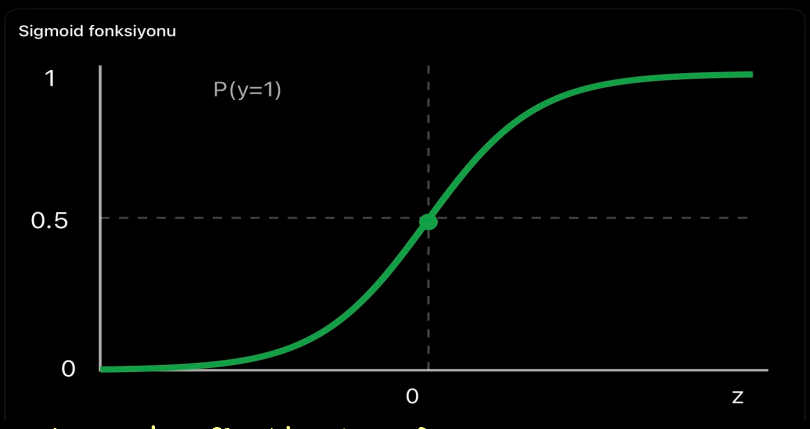
* Logistik regresyon, girdideki özelliklerin ağırlıklı toplanmasından sınıfsal olasılığı hesaplar.
* İkili sınıflandırmada sigmoid fonksiyonu kullanır:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z = w^T \cdot x + b$$

Burada:

- w: Özellik ağırlıkları
- x: Metnin sayısal vektörü
- b: Bias değeri

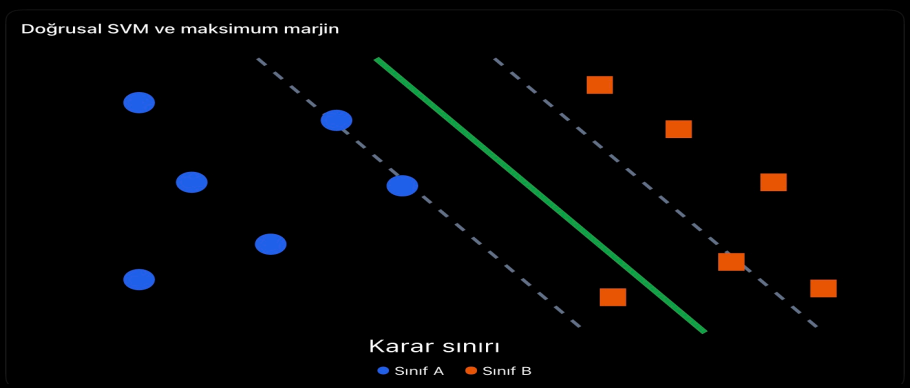


3. Suport Vector Machine (SVM)

* SVM, sınıfları ayıran en uygun karar sınırı bulmaya yarayan algoritmadır.

Temel ilkesi **maksimum marjin** yaklaşımıdır.

Marjin, karar sınırı ile en yakın eğitim örnekleri arasındaki mesafedir.



7. Derin Öğrenme ile Metin Sınıflandırma

Derin öğrenme, metinden ayırt edici özellikleri çok katmanlı sinir ağları aracılığıyla öğrenebilir.

7.1. CNN (Convolutional Neural Network)

CNN yalnızca görüntülerde değil, metin sınıflandırmada da kullanılabilir.

Metin üzerinde uygulanan filtreler, komşu kelimelerden oluşan yerel örüntüleri yakalar.

Çalışma mantığı:

Metin → Embedding → Convolution → Max Pooling → Sınıflandırma

Özellikle kısa metinlerde belirgin ifadelerin yakalanmasında etkili olabilir.

7.2. RNN, LSTM ve GRU

RNN (*Recurrent Neural Network*) kelimeleri sıralı olarak işleyen bir sinir ağıdır.

Önceki adımlardan gelen bilgileri yeni adımda kullanır.

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

- x_t: Mevcut kelime temsili
- h_t: Mevcut gizli durum
- h_{t-1}: Önceki gizli durum

RNN sorunu: Uzun metinlerde önceki bilgilerin korunması zorlaşabilir.

LSTM (Long Short-Term Memory): Kapı mekanizmalarıyla uzun süreli bağımlılıkları daha iyi yönetmeye çalışır.

GRU (Gated Recurrent Unit): LSTM'ye benzer şekilde çalışan, daha az kapı içeren tekrarlayan ağ yapısıdır.

7.3. Attention ve Self-Attention

Attention, modelin sınıflandırma yaparken girdinin farklı parçalarına farklı ağırlıklar vermesini sağlayan mekanizmadır.

Self-Attention: Bir dizideki her token'ın diğer token'larla olan ilişkisini hesaplar.

Scaled Dot-Product Attention formülü:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q: Query (sorgu)
- K: Key (anahtar)
- V: Value (değer)
- d_k: Key vektörünün boyutu

Örnek:

"Bu telefon hızlı fakat bataryası kötü."

Model, olumsuz değerlendirme açısından "fakat", "bataryası" ve "kötü" kelimeleri arasındaki ilişkiyi kullanabilir.

Attention ağırlıkları hangi token'ların ilişkili bulunduğunu gösterir; ancak tek başına kesin bir karar açıklaması sayılmaz.

7.4. Transformer ve BERT

Transformer, Attention mekanizmalarına dayalı bir derin öğrenme mimarisidir.

BERT (*Bidirectional Encoder Representations from Transformers*), iki yönlü bağlamdan yararlanarak metin temsili öğrenen Transformer tabanlı bir model ailesidir.



Fine-Tuning (İnce Ayar): Önceden eğitilmiş bir modelin belirli bir veri kümesi üzerinde yeniden eğitilerek hedef göreve uyarlanmasıdır.

Örneğin genel Türkçe metinler üzerinde eğitilmiş bir Transformer modeli haber sınıflandırma veri kümesiyle ince ayar yapılarak kullanılabilir.

8. Kayıp Fonksiyonları ve Sınıf Dengesizliği

Kayıp fonksiyonu, modelin yaptığı tahmin ile gerçek etiket arasındaki farkın sayısal ölçüsüdür.

Çok sınıflı sınıflandırmada yaygın olarak Cross-Entropy Loss kullanılır.

$$L = -\sum_{k=1}^{C} y_k \log(\hat{y}_k)$$

Burada:

- C: Sınıf sayısı
- y_k: Gerçek etiketin gösterimi
- $\hat{y}_k$: Tahmin edilen sınıf olasılığı

8.1. Dengesiz veri problemi

Örneğin 1000 metnin:

- 900'ü normal
- 100'ü spam

olsun.

Model bütün metinleri "normal" olarak tahmin ederse:

$$Accuracy = \frac{900}{1000} = 0.90$$

Yani %90 doğruluk elde edilir. Ancak model hiçbir spam mesajını tespit edemez.

Dengesizliği azaltmak için yeniden örnekleme, sınıf ağırlıkları, focal loss ve karar eşiği ayarı kullanılabilir.

9. Model Performans Değerlendirmesi

Modelin başarısını ölçmek için yalnızca doğruluk oranına bakmak yeterli değildir.

Önemli kavramlar:

- **TP (True Positive):** Doğru pozitif tahmin
- **TN (True Negative):** Doğru negatif tahmin
- **FP (False Positive):** Yanlış pozitif tahmin
- **FN (False Negative):** Yanlış negatif tahmin

9.1. Confusion Matrix (Karmaşıklık Matrisi)

	Tahmin: Pozitif	Tahmin:
Gerçek: Pozitif	TP = 40	
Gerçek: Negatif	FP = 5	

9.2. Accuracy

Bütün örnekler üzerindeki doğru tahmin oranıdır.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$= \frac{40 + 45}{100} = 0.85$$

9.3. Precision (Kesinlik)

Pozitif tahminlerin ne kadarının doğru olduğunu ölçer.

$$Precision = \frac{TP}{TP + FP}$$

$$= \frac{40}{45} \approx 0.889$$

9.4. Recall (Duyarlılık)

Gerçek pozitiflerin ne kadarının yakalandığını ölçer.

$$Recall = \frac{TP}{TP + FN}$$

$$= \frac{40}{50} = 0.80$$

9.5. F1-Score

Precision ve Recall değerlerinin harmonik ortalamasıdır.

$$F1 = 2\frac{Precision \times Recall}{Precision + Recall}$$

$$F1 \approx 0.842$$

10. Transfer Learning ve Domain Shift

Transfer Learning (Transfer Öğrenme)

Bir görevde öğrenilen bilgilerin başka bir göreve aktarılmasıdır.

Örneğin genel haber verileri üzerinde önceden eğitilen bir dil modeli, robot komutlarını sınıflandırmak için uyarlanabilir.

Domain Shift (Alan Kayması)

Modelin eğitim sırasında gördüğü veri dağılımı ile gerçek kullanım verilerinin dağılımının farklılaşmasıdır.

Örnek:

Model standart Türkçe komutlarla eğitilmiştir:

"Robot ileri hareket etsin."

Gerçek kullanıcı:

"Biraz öne gitsene."

dediğinde modelin performansı değişebilir.

Bu nedenle model farklı anlatım biçimleri, yazım hataları ve hedef kullanım ortamındaki verilerle değerlendirilmelidir.

Zero-Shot ve Few-Shot Learning

Zero-Shot: Hedef sınıfa ait görev-özel etiketli eğitim örnekleri kullanılmadan sınıflandırma.

Few-Shot: Çok az sayıda etiketli örnekten yararlanarak sınıflandırma.

Bu yaklaşımlar özellikle yeterli etiketli veri bulunmayan görevlerde önemlidir.